

Bank Loan Prediction: Data Loading, Model Training, and Confusion Matrix

This section will guide you through loading a hypothetical bank loan prediction dataset, preparing it for a machine learning model, training a simple classifier, and evaluating its performance using a confusion matrix.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Load the dataset
# IMPORTANT: Replace 'your_loan_data.csv' with the actual path to your data
# If your data is not in CSV format, adjust pd.read_csv accordingly (e.g., p

# For demonstration purposes, let's create a dummy dataset if you don't have
# In a real scenario, you would load your actual data here.
# Example of dummy data (replace this with your actual data loading):
data = {
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male',
    'Married': ['Yes', 'No', 'Yes', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No',
    'Education': ['Graduate', 'Graduate', 'Graduate', 'Not Graduate', 'Gradu
    'Self_Employed': ['No', 'No', 'Yes', 'No', 'No', 'Yes', 'No', 'Yes', 'No
    'ApplicantIncome': [5000, 4000, 6000, 3000, 7000, 4500, 5500, 8000, 3500
    'LoanAmount': [120, 100, 180, 90, 200, 110, 150, 220, 80, 190],
    'Credit_History': [1, 1, 0, 1, 1, 0, 1, 1, 1, 0],
    'Property_Area': ['Urban', 'Rural', 'Semiurban', 'Urban', 'Semiurban',
    'Loan_Status': ['Y', 'N', 'Y', 'N', 'Y', 'N', 'Y', 'Y', 'N', 'Y']
}
df = pd.DataFrame(data)

display(df.head())
print(f"Dataset shape: {df.shape}")
```

	Gender	Married	Education	Self_Employed	ApplicantIncome	LoanAmount	Cr
0	Male	Yes	Graduate	No	5000	120	
1	Female	No	Graduate	No	4000	100	
2	Male	Yes	Graduate	Yes	6000	180	
3	Female	Yes	Not Graduate	No	3000	90	
4	Male	No	Graduate	No	7000	200	

Dataset shape: (10, 9)

▼ Data Preprocessing

We'll perform some basic preprocessing steps, including handling categorical features by converting them into numerical representations and splitting the data into features (X) and target (y).

```
# Identify categorical and numerical features
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
# Exclude the target variable if it's in categorical_cols
if 'Loan_Status' in categorical_cols:
    categorical_cols.remove('Loan_Status')

# Apply Label Encoding to categorical features (for simplicity in this examp
# For more robust models, One-Hot Encoding is often preferred.
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])

# Convert the target variable 'Loan_Status' to numerical (0 or 1)
df['Loan_Status'] = df['Loan_Status'].apply(lambda x: 1 if x == 'Y' else 0)

# Define features (X) and target (y)
X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran

print("Preprocessing complete. Data split into training and testing sets.")
print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}")

Preprocessing complete. Data split into training and testing sets.
X_train shape: (7, 8), y_train shape: (7,)
X_test shape: (3, 8), y_test shape: (3,)
```

✓ Model Training

We will train a simple Logistic Regression model on the preprocessed training data.

```
# Initialize and train the Logistic Regression model
model = LogisticRegression(random_state=42, solver='liblinear') # 'liblinear'
model.fit(X_train, y_train)

print("Logistic Regression model trained successfully.")
print(f"Model accuracy on training data: {model.score(X_train, y_train):.2f}")
```

```
Logistic Regression model trained successfully.
Model accuracy on training data: 0.86
```

✓ Confusion Matrix

Finally, we will evaluate the model's performance on the test set using a confusion matrix, which shows the number of correct and incorrect predictions made by the classification model.

```
from sklearn.metrics import accuracy_score

# Calculate accuracy score
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy on Test Set: {accuracy:.2f}")
```

```
Model Accuracy on Test Set: 1.00
```

```
from sklearn.metrics import classification_report

# Generate classification report
report = classification_report(y_test, y_pred, target_names=['Not Approved',
                                                             'Approved'])

print("Classification Report:")
print(report)
```

```
Classification Report:
              precision    recall  f1-score   support

Not Approved      1.00      1.00      1.00         1
Approved          1.00      1.00      1.00         2

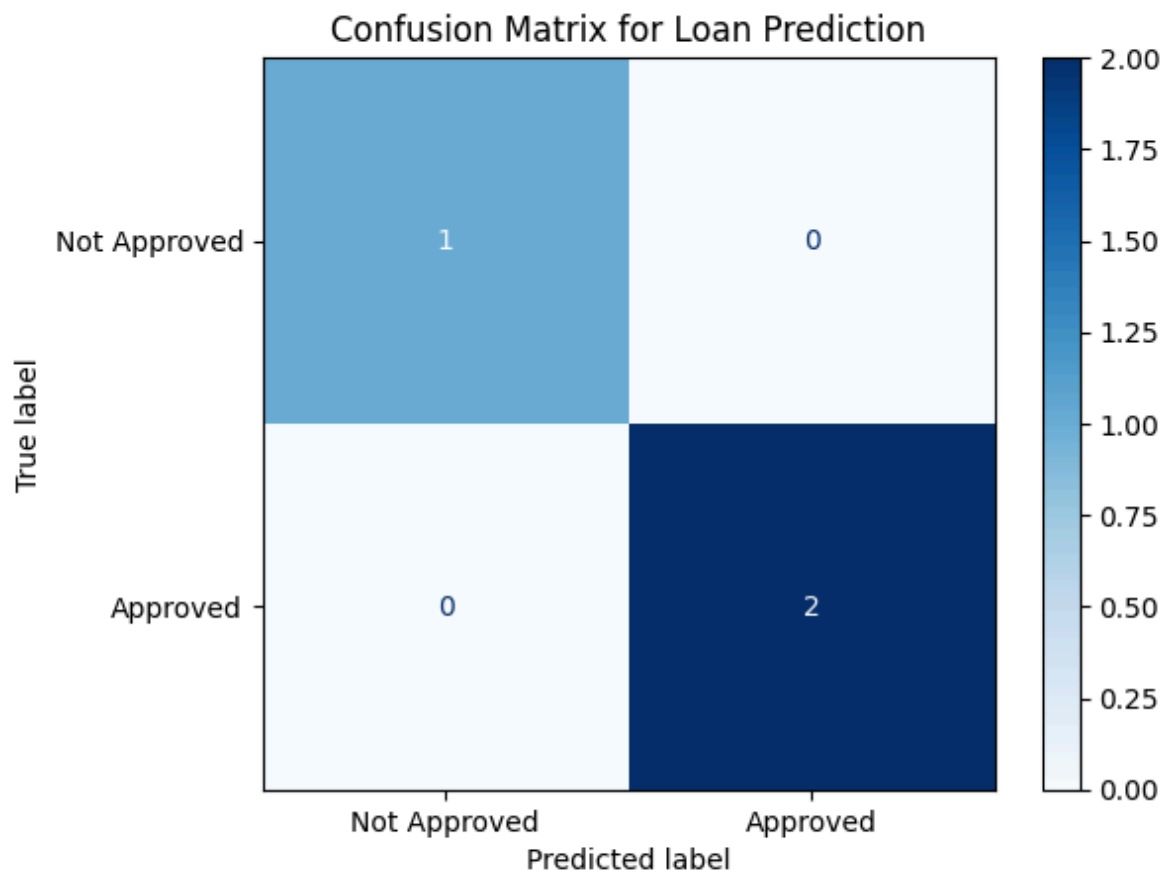
   accuracy              1.00              3
  macro avg      1.00      1.00      1.00         3
 weighted avg      1.00      1.00      1.00         3
```

```
# Make predictions on the test set
y_pred = model.predict(X_test)
```

```
# Generate the confusion matrix
cm = confusion_matrix(y_test, y_pred)

# Display the confusion matrix using ConfusionMatrixDisplay
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Not Ap
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix for Loan Prediction')
plt.show()

print("Confusion Matrix generated successfully.")
```



Confusion Matrix generated successfully.